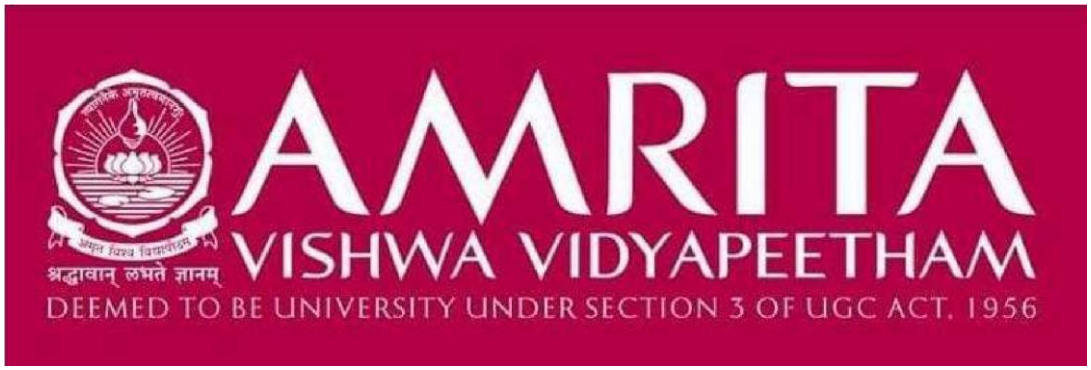


DATA STRUCTURES AND ALGORITHMS 23CSE203 LAB MANUAL

SUBMITTED BY	
NAME	Saket S
ROLL NO.	AV.SC.U4CSE25048
YEEAR/SEM/SECTION	2nd YEAR /SEM-3/CSE-A
CREDITS	05
GIT REPO LINK	https://github.com/saket0809/DSA

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(S l .Grade)
DEPARTMENT	School of Computing

.

	TITLE	PAGE NO	DATE
1	Lab1:- 1a) rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch. 1b)Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years. 1c) An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. 1d) In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number. 1e) Write a recursive Python program to print the first n terms of a Fibonacci series.	3-12	23-07-26
2	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	13-16	30-07-26

.

3.	WEEK 3: 3a. Write a program to perform Bubble Sort technique on the user given input. 3b. Write a program to perform Insertion Sort technique on the user given input. 3c. Write a program to perform Selection Sort technique on the user given input. 3d. Write a program to perform Merge Sort technique on the user given input. 3e. Write a program to perform Quick Sort technique on the user given input	16-25	06-08-26
4	WEEK 4: 4a. Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user. a. Create a linked list. b. Insert at beginning. c. Insert at end. d. Insert at a specific index. e. Delete by value. f. Delete 1st node. g. Delete last node. h. Count no. of nodes i. Display. j. Exit. 4b.Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user. 4c. Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user. Also add these two options accordingly. 1. Display head and tail. 2. Print data from tail to head.	26-66	13-08-26
5	WEEK 5: Write a python program to implement a stack using an a)Array, b)Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	66-74	13-08-26
6)	WEEK 6: To implement Queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu driven approach. 6a. Using Array, Linked list, Data structures. 6b. Circular Queue.	74-86	03-09-26
7)	WEEK 7: Write a python program to implement binary tree 7a. Using Array, and 7b. Linked List With all of its operation.	86-100	24-09-26

.

8)	WEEK 8: Write a python program to implement Binary Search tree with its operation.	100-109	08-10-26
9)	WEEK-9: Write a python program to implement AVL Tree and its operations.	110-119	22-10-26
10)	WEEK-10: To implement hashing using separate chaining, linear and quadratic probing(using a)array and b)linked list)	120-131	29-10-26

.

LAB-1

1A)AIM:- rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch.

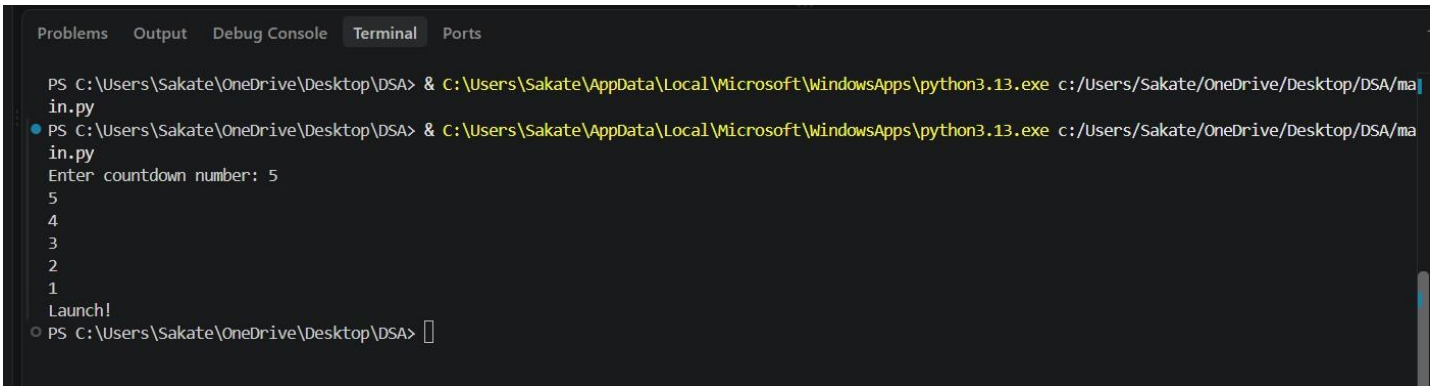
PROGRAM:-

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

# Input
n = int(input("Enter countdown number: "))

# Function call
countdown(n)
```

OUTPUT:-



EXPLANATION:-

.

- def defines the countdown() function.
- input() accepts the countdown value from the user.
- int() converts the input into an integer.
- countdown() starts the recursive countdown process.
- if checks the base case (n == 0).
- else executes when the condition is False.
- print(n) displays the current countdown number.
- countdown(n - 1) recursively calls the function with the value decreased by 1.
- print("Launch!") displays the launch message when the countdown reaches 0.
- The program demonstrates the use of functions, recursion, conditional statements, input, and output to perform the countdown.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Missing base case (if n == 0) causing infinite recursion.	Include the base case to stop recursion .l
3	incorrect indentation leading to an Indentation Error	Maintain proper Python indentation.
4	Missing colon (:) after def, if, or else.	Add the required colon (:) after statements.
5	Calling countdown(n) instead of countdown(n - 1), causing infinite recursion.	Decrease the value by 1 in each recursive call using countdown(n - 1).

TIME COMPLEXITY:- O(n)

SPACE COMPLEXITY:- O(n)

REASON:- Function is called once for each value from n to 0.

.

1B)AIM:- Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years.

PROGRAM:-

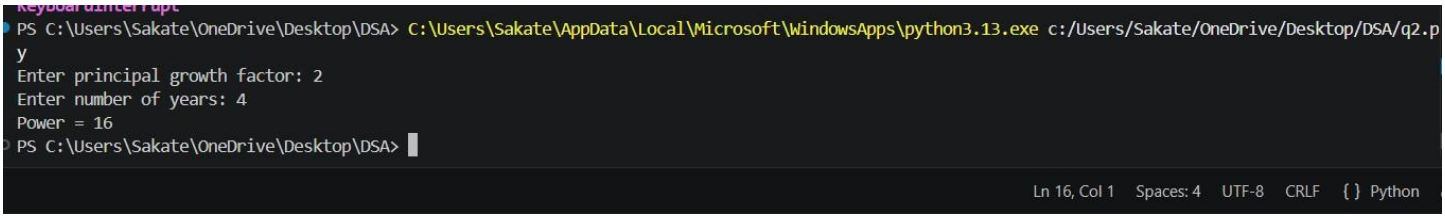
```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

# Input
p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

# Function call
result = power(p, n)

# Output
print("Power =", result)
```

OUTPUT:-



.

EXPLANATION:-

- power() is the recursive function that computes P raised to n.
- The base case is n == 0, which returns 1 (any number to the power 0 is 1).
- The recursive case multiplies p with power(p, n - 1), reducing n by 1 on each call.
- float() allows the principal growth factor to be a decimal value.
- input() collects the values of P and n from the user.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the final computed value of P^n.

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base case (n == 0) causing infinite recursion.	Include the base case to stop recursion.
2	Entering a negative value for n causes the recursion to never reach 0.	Restrict input to non-negative integers or handle negative n separately.
3	Using int() for P truncates decimal growth factors.	Use float() to accept decimal values for P.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Calling power(p, n) instead of power(p, n - 1), causing infinite recursion.	Decrease n by 1 in each recursive call using power(p, n - 1).

TIME COMPLEXITY:- O(n)

SPACE COMPLEXITY:- O(n)

REASON:- Recursive call is made n times to compute P^n.

.

1C)AIM:- An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:-

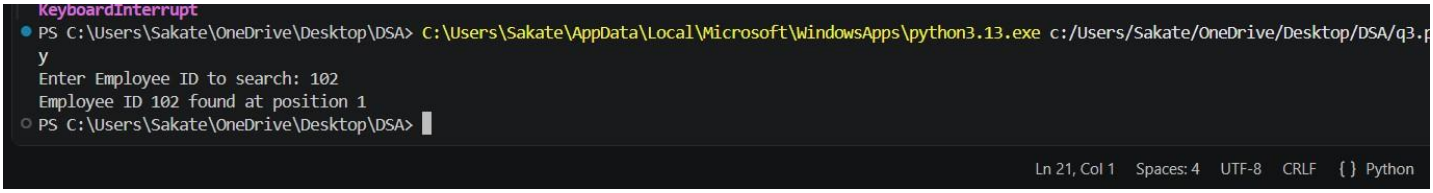
```
def search_employee(emp_list, emp_id, index=0):
    if index == len(emp_list):
        return -1
    elif emp_list[index] == emp_id:
        return index
    else:
        return search_employee(emp_list, emp_id, index + 1)


# Input
employee_ids = [101, 102, 103, 104, 105]
search_id = int(input("Enter Employee ID to search: "))

# Function call
position = search_employee(employee_ids, search_id)

if position != -1:
    print("Employee ID", search_id, "found at position", position)
else:
    print("Employee ID", search_id, "not found")
```

OUTPUT:-



EXPLANATION:-

.

- search_employee() is the recursive function that searches the list one index at a time.
- The base case index == len(emp_list) means every element was checked and the ID was not found, so it returns -1.
- The second condition checks if the current element matches the employee ID being searched.
- If not matched, the function calls itself with index + 1, moving to the next employee in the list.
- employee_ids is the list of employee IDs stored by the HR department.
- input() and int() collect the employee ID to search for from the user.
- The if-else block reports whether the employee ID was found and at which position.

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base case (index == len(emp_list)) causing infinite recursion.	Include the base case to stop recursion when the list is exhausted.
2	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
3	Calling search_employee(emp_list, emp_id, index) without incrementing index, causing infinite recursion.	Increase index by 1 in each recursive call using index + 1.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Searching an empty list without handling it, giving a confusing result.	Add a check for an empty list before searching.

TIME COMPLEXITY:- O(n)

SPACE COMPLEXITY:- O(n)

REASON:- In the worst case, every element in the list is checked recursively.

.

1D)AIM:- In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number.

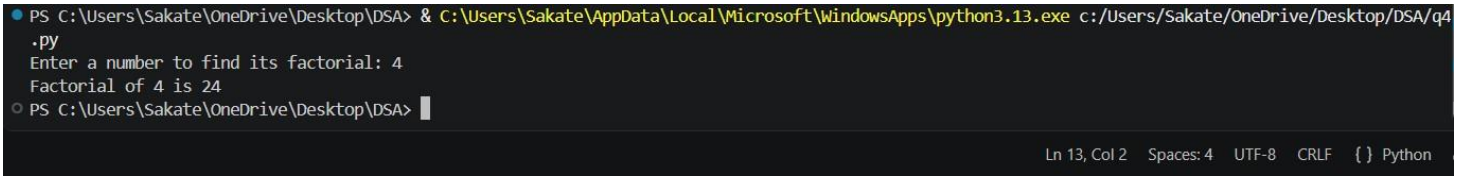
PROGRAM:-

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


# Input
n = int(input("Enter a number to find its factorial: "))

# Function call
print("Factorial of", n, "is", factorial(n))
```

OUTPUT:-



EXPLANATION:-

- factorial() is the recursive function that computes n! (n factorial).
- The base case is n == 0 or n == 1, both of which return 1.
- The recursive case multiplies n with factorial(n - 1), reducing n by 1 on each call.
- input() and int() collect the number whose factorial is to be found.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the number of possible arrangements (the factorial value).

ERRORS:-

.

SNO	Errors Observed	Rectification
1	Missing base case (n == 0 or n == 1) causing infinite recursion.	Include the base case to stop recursion.
2	Entering a negative number, for which factorial is undefined.	Restrict input to non-negative integers.
3	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Calling factorial(n) instead of factorial(n - 1), causing infinite recursion.	Decrease n by 1 in each recursive call using factorial(n - 1).

TIME COMPLEXITY:- O(n)

SPACE COMPLEXITY:- O(n)

REASON:- Function calls itself n times until reaching the base case.

.

1E)AIM:- Write a recursive Python program to print the first n terms of a Fibonacci series.

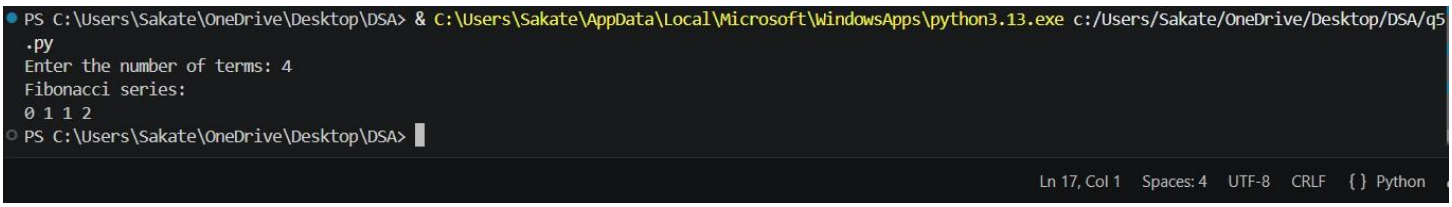
PROGRAM:-

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


# Input
n = int(input("Enter the number of terms: "))

# Function call
print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:-



EXPLANATION:-

- fibonacci() is the recursive function that returns the nth Fibonacci term.
- The base cases are n == 0 (returns 0) and n == 1 (returns 1).
- The recursive case adds fibonacci(n - 1) and fibonacci(n - 2), following the Fibonacci rule.
- input() and int() collect the number of terms to be printed.
- The for loop calls fibonacci(i) for each index from 0 to n - 1.
- print(..., end=" ") displays each term on the same line, separated by a space.

.

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base cases (n == 0 and n == 1) causing infinite recursion.	Include both base cases to stop recursion.
2	Entering a negative number of terms.	Restrict input to non-negative integers.
3	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Using fibonacci(n - 1) - fibonacci(n - 2) instead of addition, giving wrong terms.	Use addition, fibonacci(n - 1) + fibonacci(n - 2), as per the Fibonacci rule.

TIME COMPLEXITY:- $O(2^n)$

SPACE COMPLEXITY:- O(n)

REASON:- Each call creates two recursive calls, resulting in exponential time, while the recursion depth is n.

.

LAB-2

2A)AIM:- Write a Python program to perform linear search for the given input.

PROGRAM:-

```
def linear_search(arr, target):
for index in range(len(arr)):
if arr[index] == target:
        return index
return -1

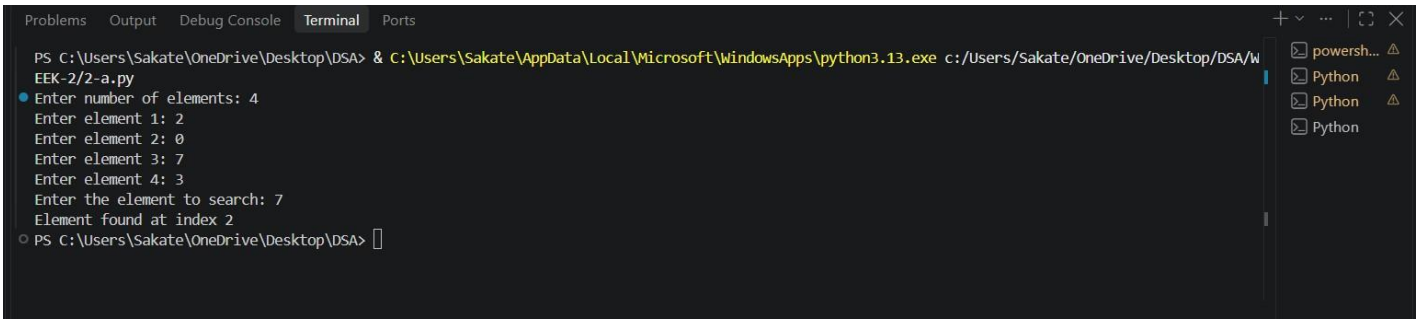

# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
   arr.append(int(input(f"Enter element {i+1}: ")))

target = int(input("Enter the element to search: "))

# Function call result =
linear_search(arr, target)

# Output
if result != -1:
   print("Element found at index",
result) else:    print("Element not found
in the list")
```

OUTPUT:-



.

EXPLANATION:-

- linear_search() checks each element of the list one by one from the beginning.
- The for loop iterates over every index of the array using range(len(arr)).
- If arr[index] equals the target, the matching index is returned immediately.
- If the loop finishes without a match, -1 is returned to indicate the element is not present.
- input() and int() collect the number of elements, the elements themselves, and the target value.
- The if-else block reports whether the target was found and at which index, or that it was not found.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Searching an empty list (n = 0) without handling it.	Add a check for an empty list before searching.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Using == to compare elements of different types (e.g. int vs str), causing a false "not found".	Ensure the target and list elements are converted to the same type (int) before comparing.

TIME COMPLEXITY:- O(n)

SPACE COMPLEXITY:- O(1)

REASON:- In the worst case, every element in the list is checked once; no extra space is used besides a few variables

.

2B)AIM:- Write a Python program to perform binary search of the given input, verifying by taking a sorted array and an unsorted array as input.

PROGRAM:-

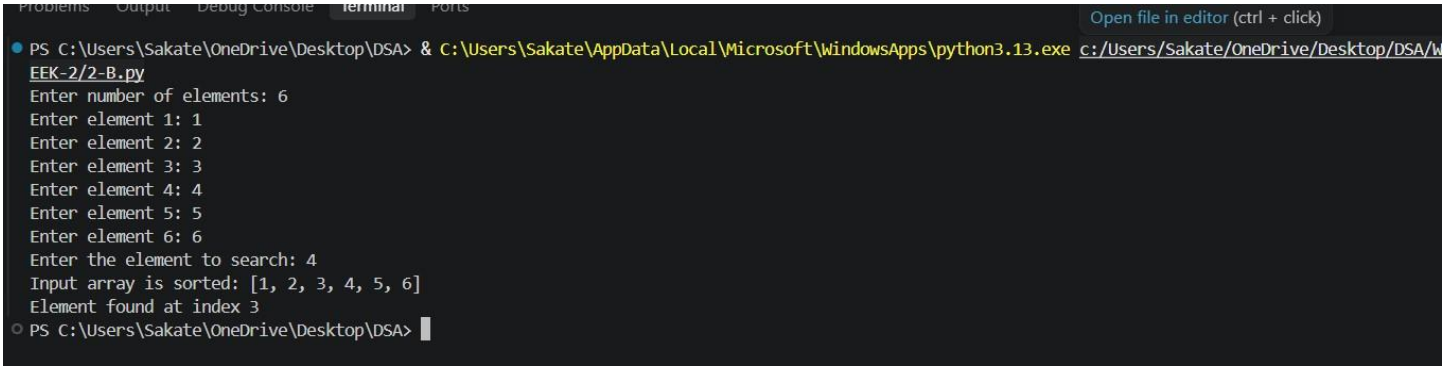
```
def binary_search(arr, target):
   low = 0     high =
len(arr) - 1     while
low <= high:
       mid = (low + high) // 2
if arr[mid] == target:
           return mid
elif arr[mid] < target:
           low = mid +
1         else:
high = mid - 1
return -1


# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
   arr.append(int(input(f"Enter element {i+1}: ")))


target = int(input("Enter the element to search: "))


# Verify whether the input array is sorted or unsorted
if arr == sorted(arr):
   print("Input array is sorted:", arr)
result = binary_search(arr, target)
if result != -1:
       print("Element found at index",
result)     else:         print("Element not
found in the list") else:
   print("Input array is NOT sorted:", arr)     print("Binary search requires a
sorted array.")     print("Result on unsorted array cannot be trusted. Sorting
the array first...")     sorted_arr = sorted(arr)     print("Sorted array:",
sorted_arr)     result = binary_search(sorted_arr, target)     if result != -1:
       print("Element found at index", result, "in the sorted
array")     else:         print("Element not found in the list")
```

OUTPUT:-



EXPLANATION:-

- binary_search() repeatedly narrows the search range between low and high using the midpoint mid.
- If arr[mid] equals the target, the index mid is returned immediately.
- If arr[mid] is less than the target, the search continues in the right half by moving low to mid + 1.
- Otherwise, the search continues in the left half by moving high to mid - 1.
- arr == sorted(arr) checks whether the entered array is already sorted, to verify binary search's precondition.
- If the array is sorted, binary search runs directly on it and reports the result.
- If the array is unsorted, the program shows that binary search cannot be trusted on it, sorts the array with sorted(), and re-runs binary search on the sorted copy.

.

ERRORS:-

SNO	Errors Observed	Rectification
1	Running binary search directly on an unsorted array gives an unreliable or wrong result.	Always verify with arr == sorted(arr) and sort the array before searching.
2	Using mid = (low + high) / 2 gives a float, causing a TypeError when indexing the list.	Use integer floor division (//) instead of / to compute mid.
3	Using low < high instead of low <= high, missing the case where low equals high.	Use low <= high so a single remaining element is still checked.
4	Not updating low or high inside the loop, causing an infinite loop.	Always set low = mid + 1 or high = mid - 1 inside the loop body.
5	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.

TIME COMPLEXITY:- O(log n)

SPACE COMPLEXITY:- O(1)

REASON:- The search range is halved on every iteration, so the number of comparisons grows logarithmically with n; only a few variables (low, high, mid) are used regardless of input size.

.

LAB-3

3A)AIM:- Write a Python program to bubble sort the elements, taking input from the user.

PROGRAM:-

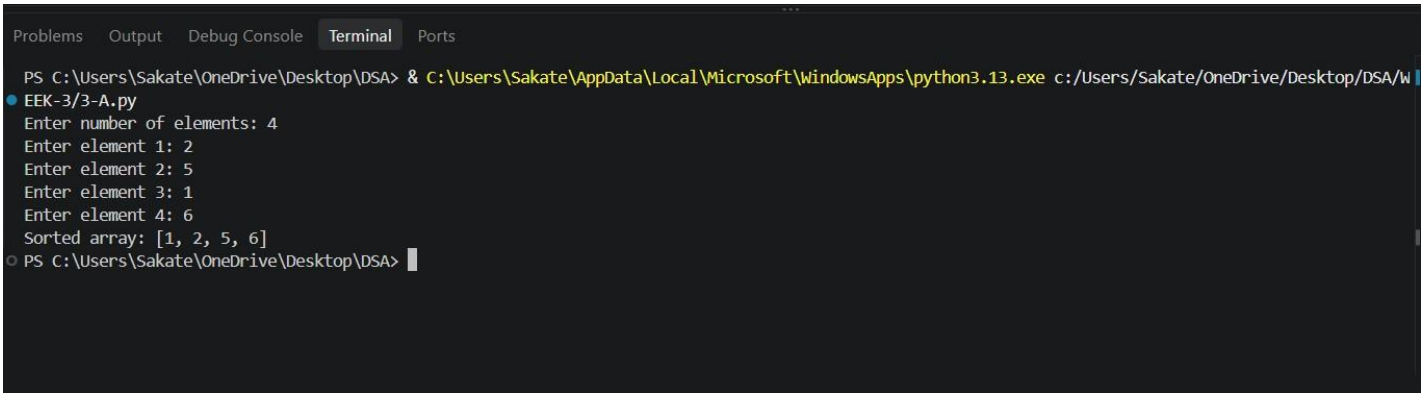
```
def bubble_sort(arr):      n
= len(arr)     for i in
range(n - 1):         for j in
range(n - 1 - i):             if
arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
return arr


 # Input n = int(input("Enter number of elements: ")) arr = [] for i in range(n):
   arr.append(int(input(f"Enter element {i+1}: ")))


# Function call sorted_arr = bubble_sort(arr)


 # Output
print("Sorted array:", sorted_arr)
```

OUTPUT:-



.

EXPLANATION:-

- bubble_sort() repeatedly steps through the list, comparing adjacent elements.
- The outer loop runs n - 1 times, controlling the number of passes needed.
- The inner loop compares arr[j] and arr[j + 1], swapping them if they are out of order.
- After each pass, the largest remaining unsorted element "bubbles up" to its correct position at the end.
- input() and int() collect the number of elements and each element from the user.
- print() displays the final sorted array.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Using range(n) instead of range(n - 1 - i) in the inner loop, causing an IndexError.	Use range(n - 1 - i) so arr[j + 1] never goes out of bounds.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Swapping elements incorrectly without simultaneous assignment, losing a value.	Use arr[j], arr[j + 1] = arr[j + 1], arr[j] for a correct swap.

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- O(1)

REASON:- Two nested loops compare and swap every pair of elements in the worst case; sorting is done in place using only a few extra variables.

.

3B)AIM:- Write a Python program to insertion sort the elements, taking input from the user.

PROGRAM:-

```
def insertion_sort(arr):
for i in range(1, len(arr)):
      key = arr[i]        j = i - 1
while j >= 0 and arr[j] > key:
          arr[j + 1] =
arr[j]            j -= 1
arr[j + 1] = key
return arr



# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
   arr.append(int(input(f"Enter element {i+1}: ")))



# Function call sorted_arr =
insertion_sort(arr)



# Output
print("Sorted array:", sorted_arr)
```

OUTPUT:-



EXPLANATION:-

- insertion_sort() builds the sorted list one element at a time, similar to sorting playing cards in hand.

.

- key stores the current element that is being inserted into the sorted portion of the array.
- The while loop shifts elements greater than key one position to the right.
- j decreases until the correct position for key is found or the start of the array is reached.
- arr[j + 1] = key places the element in its correct sorted position.
- input() and int() collect the number of elements and each element from the user.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Missing the boundary check j >= 0, causing an IndexError.	Always check j >= 0 before comparing arr[j] inside the while loop.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Forgetting to decrement j inside the while loop, causing an infinite loop.	Decrease j by 1 in each iteration using j -= 1.

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- O(1)

REASON:- In the worst case (reverse sorted input), every element is compared against and shifted through the entire sorted portion; sorting is done in place.

.

3C)AIM:- Write a Python program to selection sort the elements, taking input from the user.

PROGRAM:-

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr


# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = selection_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

.

OUTPUT:-



EXPLANATION:-

- **selection_sort() repeatedly selects the minimum element from the unsorted part and places it at the beginning.**
- **min_index tracks the index of the smallest element found during the current pass.**
- **The inner loop scans the unsorted portion of the array to find the position of the minimum value.**
- **arr[i], arr[min_index] = arr[min_index], arr[i] swaps the found minimum into its correct sorted position.**
- **input() and int() collect the number of elements and each element from the user.**
- **print() displays the final sorted array.**

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.

.

3	**Not resetting min_index to i at the start of each outer pass, giving a wrong sort.**	**Reset min_index = i at the beginning of every outer loop iteration.**
4	**Incorrect indentation leading to an IndentationError.**	**Maintain proper Python indentation.**
5	**Using <= instead of < when comparing arr[j] and arr[min_index], causing unnecessary swaps.**	**Use strict < so equal elements are not swapped needlessly.**

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- O(1)

REASON:- The inner loop scans all remaining unsorted elements for every outer iteration, giving n^2 comparisons; sorting is done in place.

.

3D)AIM:- Write a Python program to merge sort the elements, taking input from the user.

PROGRAM:-

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Input
n = int(input("Enter number of elements: "))
```

.

```
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = merge_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

OUTPUT:-



EXPLANATION:-

- **merge_sort() recursively splits the array into halves until each sub-array has one element.**
- **mid = len(arr) // 2 finds the midpoint used to divide the array into left and right halves.**
- **left and right hold the recursively sorted halves of the array.**
- **merge() combines two sorted halves into a single sorted list by comparing their front elements.**
- **result.extend(...) appends any elements left over once one half is fully consumed.**

.

- **input() and int() collect the number of elements and each element from the user.**

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Missing base case (len(arr) <= 1), causing infinite recursion.	Include the base case to stop recursion when the sub-array has one or zero elements.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Forgetting to extend the remaining elements after the while loop in merge(), losing elements.	Use result.extend(left[i:]) and result.extend(right[j:]) after the loop.

TIME COMPLEXITY:- O(n log n)

SPACE COMPLEXITY:- O(n)

REASON:- The array is divided log n times and each level requires O(n) work to merge the halves; additional arrays are created at every recursive call.

.

3E)AIM:- Write a Python program to quick sort the elements, taking input from the user.

PROGRAM:-

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)


# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = quick_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

.

OUTPUT:-

```
Enter number of elements: 4
Enter element 1: 13
Enter element 2: 23
Enter element 3: 27
Enter element 4: 89
Sorted array: [13, 23, 27, 89]
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- **quick_sort() recursively partitions the array around a chosen pivot element.**
- **pivot = arr[len(arr) // 2] selects the middle element of the array as the pivot for partitioning.**
- **left, middle, and right use list comprehensions to group elements less than, equal to, and greater than the pivot.**
- **The function recursively sorts left and right and concatenates them with middle in between.**
- **input() and int() collect the number of elements and each element from the user.**
- **print() displays the final sorted array.**

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.

.

3	**Missing base case (len(arr) <= 1), causing infinite recursion.**	**Include the base case to stop recursion when the sub-array has one or zero elements.**
4	**Incorrect indentation leading to an IndentationError.**	**Maintain proper Python indentation.**
5	**Always choosing the first element as pivot on already-sorted input, degrading performance to $O(n^2)$.**	**Choose a better pivot strategy, such as the middle element or a random element.**

TIME COMPLEXITY:- O(n log n) average, $O(n^2)$ worst case

SPACE COMPLEXITY:- O(log n)

REASON:- On average the pivot splits the array into two roughly equal halves, giving log n levels of recursion with O(n) work per level; the recursion stack depth is O(log n) on average.

.

LAB-4

AIM:- Write a Python program to implement a singly linked list that performs the following operations, taking input from the user: create a linked list, insert at the beginning, insert at the end, insert at a specific index, delete by value, delete the first node, delete the last node, count the number of nodes, display/traverse the list, and exit.

PROGRAM:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create_list(self):
        self.head = None
        n = int(input("Enter number of nodes to create: "))
        for i in range(n):
            data = int(input(f"Enter value for node {i + 1}: "))
            self.insert_at_end(data)
        print("Linked list created successfully.")

    def insert_at_beginning(self):
        data = int(input("Enter value to insert at beginning: "))
```

.

```
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    print(f"{data} inserted at the beginning.")

def insert_at_end(self, data=None):
    if data is None:
        data = int(input("Enter value to insert at end: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    if data is not None:
        print(f"{data} inserted at the end.")

def insert_at_index(self):
    index = int(input("Enter index to insert at (0-based): "))
    data = int(input("Enter value to insert: "))
    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"{data} inserted at index {index}.")
        return
    temp = self.head
    count = 0
    while temp is not None and count < index - 1:
```

.

```
            temp = temp.next
            count += 1
        if temp is None:
            print("Index out of range.")
            return
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
        print(f"{data} inserted at index {index}.")

    def delete_by_value(self):
        value = int(input("Enter value to delete: "))
        if self.head is None:
            print("List is empty.")
            return
        if self.head.data == value:
            self.head = self.head.next
            print(f"Value {value} deleted.")
            return
        prev = self.head
        curr = self.head.next
        while curr:
            if curr.data == value:
                prev.next = curr.next
                print(f"Value {value} deleted.")
                return
            prev = curr
            curr = curr.next
        print(f"Value {value} not found in list.")

    def delete_first_node(self):
```

.

```
    if self.head is None:
        print("List is empty.")
        return
    deleted = self.head.data
    self.head = self.head.next
    print(f"Deleted first node with value {deleted}.")

def delete_last_node(self):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.next is None:
        deleted = self.head.data
        self.head = None
        print(f"Deleted last node with value {deleted}.")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    deleted = temp.next.data
    temp.next = None
    print(f"Deleted last node with value {deleted}.")

def count_nodes(self):
    count = 0
    temp = self.head
    while temp:
        count += 1
        temp = temp.next
    print(f"Number of nodes: {count}")
```

```
    .

    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        elements = []
        while temp:
            elements.append(str(temp.data))
            temp = temp.next
        print(" -> ".join(elements) + " -> None")


# Main menu-driven program
sll = SinglyLinkedList()

while True:
    print("\n----- Singly Linked List Menu       ")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display/Traversal")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
```

```
        sll.create_list()
    elif choice == 2:
        sll.insert_at_beginning()
    elif choice == 3:
        sll.insert_at_end()
    elif choice == 4:
        sll.insert_at_index()
    elif choice == 5:
        sll.delete_by_value()
    elif choice == 6:
        sll.delete_first_node()
    elif choice == 7:
        sll.delete_last_node()
    elif choice == 8:
        sll.count_nodes()
    elif choice == 9:
        sll.display()
    elif choice == 10:
        print("Exiting program.")
        break
    else:
        print("Invalid choice. Try again.")
```

OUTPUT:-

```
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 1
Enter number of nodes to create: 5
Enter value for node 1: 23
23 inserted at the end.
Enter value for node 2: 27
27 inserted at the end.
Enter value for node 3: 32
32 inserted at the end.
Enter value for node 4: 73
73 inserted at the end.
Enter value for node 5: 89
89 inserted at the end.
Linked list created successfully.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 2
Enter value to insert at beginning: 1
1 inserted at the beginning.
```

```
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 3
Enter value to insert at end: 2
2 inserted at the end.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 4
Enter index to insert at (0-based): 3
Enter value to insert: 3
3 inserted at index 3.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
```

```
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 6
Deleted first node with value 1.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 7
Deleted last node with value 2.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 8
```

```
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 8
Number of nodes: 5

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 9
23 -> 27 -> 32 -> 73 -> 89 -> None

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 10
Exiting program.
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

.

EXPLANATION:-

- The Node class stores a data value and a reference (next) to the following node in the list.
- The SinglyLinkedList class maintains a head pointer and provides methods for each menu operation.
- create_list() resets the list and repeatedly calls insert_at_end() to build it from user input.
- insert_at_beginning() creates a new node, points it to the current head, and makes it the new head.
- insert_at_end() traverses to the last node and links a new node after it, or sets it as head if the list is empty.
- insert_at_index() walks to the node just before the given position and relinks pointers to insert the new node there.
- delete_by_value() searches the list for a matching value and removes that node by updating the previous node's next pointer.
- delete_first_node() and delete_last_node() move the head pointer or unlink the final node after locating it by traversal.
- count_nodes() and display() both traverse the list from head to end, counting nodes or collecting their values.
- The main program runs an infinite while loop that prints the menu, reads the user's choice, and calls the matching method until option 10 (Exit) breaks the loop.

ERRORS:-

.

SNO	Errors Observed	Rectification
1	Entering a non-integer value (e.g. letters) at any input prompt causes a ValueError.	Enter a valid integer whenever the program asks for a value, index, or choice.
2	Choosing an operation such as display, delete, or count before creating the list results in the 'List is empty.' message.	Select option 1 to create the list first before using the other operations.
3	Entering an index beyond the current size in 'Insert at specific index' prints 'Index out of range.'	Check the current node count (option 8) and enter an index within range.
4	Trying to delete a value that is not present in the list prints 'Value not found in list.'	Verify the value exists by displaying the list (option 9) before deleting it.
5	Entering a choice outside 1-10 shows 'Invalid choice. Try again.'	Re-enter a number between 1 and 10 from the menu.

TIME COMPLEXITY:- Insert at beginning and delete first node are O(1); insert at end/index, delete by value/last node, count, and display are O(n)

SPACE COMPLEXITY:- O(n) for storing n nodes, O(1) auxiliary space per operation

.

REASON:- Operations that only touch the head (insert at beginning, delete first node) run in constant time because no traversal is needed, while operations that must reach a specific position, value, or the end of the list (insert at end/index, delete by value/last node, count, display) require walking through up to n nodes in the worst case.

.

Lab-5

1.Aim: Write a python program to implement a stack using an Array. Implement the stack operations such as push, pop, peak and also display the stack elements. PROGRAM: Stack using an Array

Program:

```
# Stack Implementation using Array


class Stack:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, data):
        self.stack.append(data)
        print(data, "pushed into stack.")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow.")
        else:
            data = self.stack.pop()
```

```
        print(data, "popped from stack.")

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element:", self.stack[-1])

    # Display stack
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


# Main Program
s = Stack()
```

.

```
while True:
    print("\n========== STACK USING ARRAY ==========")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=========================================")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()
```

.

```
    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT :

```
========== WEEK-5 ==========
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice: 1

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 23
Pushed: 23

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 27
Pushed: 27

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
```

.

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
WEEK 5/stack.py"
--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped: 89

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: ['23', '27']

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4

========== WEEK-5 ==========
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice:
```

Explanation:

1.stack = [] creates an empty list which is used as the array of the stack.

2.size stores the maximum number of elements the stack can hold, entered by the user.

.

3. The push(item) function first checks whether len(stack) == size. This is the overflow condition.
4. If the stack is full, the program prints "Stack Overflow" and the element is not inserted.
5. Otherwise, stack.append(item) inserts the element at the top of the stack.
6. The user enters the number of elements to push, and a for loop reads each element and calls push().
7. Finally, the stack is displayed from top to bottom using stack[::-1].

Error table:

Si.no	**Error obtained**	**Rectification**
1.	if len(stack) = size:	if len(stack) == size:

.

2.Aim: Write a python program to implement a stack using an Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program: Stack using an Linked List.

```
# Stack Implementation using Linked List


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into stack.")
```

.

```
# Pop operation
def pop(self):
    if self.top is None:
        print("Stack Underflow.")
    else:
        data = self.top.data
        self.top = self.top.next
        print(data, "popped from stack.")

# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty.")
    else:
        print("Top element:", self.top.data)

# Display stack
def display(self):
    if self.top is None:
        print("Stack is empty.")
```

```
    .
        else:
            temp = self.top
            print("Stack elements:")

            while temp is not None:
                print(temp.data)
                temp = temp.next


# Main Program
s = Stack()

while True:
    print("\n======= STACK USING LINKED LIST =======")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=======================================")
```

```
.
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter data: "))
    s.push(data)

elif choice == 2:
    s.pop()

elif choice == 3:
    s.peek()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

.

OUTPUT:

```
========== WEEK-5 ==========
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice: 2

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 23
Pushed: 23

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 27
Pushed: 27

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Linked List Stack ---
1. Push
2. Pop
```

·

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped: 89

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: 27 23

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4

========== WEEK-5 ==========
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice:
```

EXPLANATION:

Node(data) : Creates a node and stores the element in data. ·

next = None : Initially, the node has no link. ·

top = None : Initializes an empty stack.

·

Push:

· new_node = Node(data) : Creates a new node. ·

new_node.next = self.top : Links it to the current top. ·

self.top = new_node : Makes it the new top.

· Process: Create : Link : Update top.

Pop: ·

Checks self.top is None for underflow. ·

popped = self.top.data : Gets the top value. ·

self.top = self.top.next : Moves top to the next node. ·

Process: Get value : Move top : Remove old top.

Peek:

· self.top.data : Displays the top value without removing it.

Display:

· temp = self.top : Starts from top. ·

while temp : Traverses the nodes.

· temp = temp.next : Moves to the next node.

.

Main Program:

· SLL = Stack LinkedList() : Creates the stack. ·

choice : Selects Push, Pop, Peek, Display, or Exit. ·

while True : Repeats the menu. ·

break : Exits the program. ·

LIFO: Last inserted : First removed.

ERROR TABLE:

s.no	Error Obtained	Rectification
1.	Attribute Error may occur in Linked List Stack if self.top.data is accessed when top is None.	Check if self.top is None before performing pop() or peek().

.

WEEK - 6

6a)Aim: To write a Python program to implement a queue using an array (list) and linked list and perform insertion (Enqueue), deletion (Dequeue), Peek, and Display operations using the FIFO (First In, First Out) principle.

Program: (Queue using array).

```
queue=[] size=int(input("Enter the size of the
queue:"))


while True:
   print(" \nQueue Menu:")       print("1.Enqueue")
print("2.Dequeue")                    print("3.Peek")
print("4.Display")     print("5.Exit")


   choice=int(input("Enter your choice form 1-5:"))


   if  choice==1:                  if
len(queue)==size:
         print("Queue is full.Cant add elements into the queue")
else:
         value=int(input("Enter the value to be added into the queue:"))
queue.append(value)            print("Value inserted")
```

```
    .


   elif choice==2:
      if len(queue)==0:
         print("There are no elements to delete in the queue")
else:
         value=queue.pop(0)
print("Value deleted:",value)


   elif choice==3:          if
len(queue)==0:
         print("There are no elements no print in the queue")
else:
         print("First element is:",queue[0])


   elif choice==4:          if
len(queue)==0:
         print("There are no elements to display")
else:
         print("Queue:",queue)
```

.

```
elif choice==5:         print("Exiting...")

 else:

   print("invalid choice")
```

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/W
EEK-6/QueueArray.py
5. Count
6. Exit
Enter your choice: 1
Enter the element: 12
12 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 23
23 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 27
27 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Queue elements are:
12
23
27
```

.

Explanation:

Explanation:

1. Queue Initialization: An empty list queue = [] is created to store elements, and size = 5 specifies the maximum queue capacity.

2. Menu Display: A while True loop displays the menu options: Enqueue, Dequeue, Peek, Display, and Exit.

3. Enqueue (Insertion): If the queue is not full, the program accepts an element from the user and inserts it at the rear using append().

4. Dequeue (Deletion): If the queue is not empty, the program removes and displays the front element using pop(0).

5. Peek Operation: The program displays the front element using queue[0] without removing it.

6. Display Operation: The program prints all the elements present in the queue. If the queue is empty, it displays a message.

7. Exit Operation: When the user selects option 5, the break statement terminates the loop and ends the program.

8. Invalid Choice: If the user enters an option other than 1 to 5, the program displays "Invalid choice".

.

Error Tables:

S.No	Errors obtained	Rectification
01.	Queue Overflow	Occurs when inserting an element into a full queue. Check whether len(queue) == size before insertion.
02.	Index Error	Occurs when accessing queue[0] while the queue is empty. Check whether the queue is empty before accessing the front element.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
```

.

```
# Enqueue operation
def enqueue(self, data):
    new_node = Node(data)

    if self.rear is None:
        self.front = new_node
        self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node

    print(data, "inserted into the queue.")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow. Queue is empty.")
    else:
        data = self.front.data
        self.front = self.front.next
```

.

```
        if self.front is None:
            self.rear = None

        print(data, "deleted from the queue.")

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element is:", self.front.data)

# Display all elements
def display(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Queue elements are:")

        temp = self.front
```

```
    .
        while temp is not None:
            print(temp.data)
            temp = temp.next

    # Count number of elements
    def count(self):
        count = 0
        temp = self.front

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of elements:", count)



# Main program
q = Queue()

while True:
    print("\n--- QUEUE MENU ---")
```

```
print("1. Enqueue")
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Count")
print("6. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter the element: "))
    q.enqueue(data)

elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()
```

.

```
elif choice == 5:
    q.count()

elif choice == 6:
    print("Program ended.")
    break

else:
    print("Invalid choice.")
```

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA
EEK-6/QueueArray.py
KeyboardInterrupt
PS C:\Users\Sakate\OneDrive\Desktop\DSA> C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/W
K-6/QueueLL.py

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 23
23 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 27
27 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 07
7 inserted into the queue.
```

Explanation:

1. Node Creation: The Node class stores the element in data and the address of the next node in next.

2. Initialization: The variables front and rear are initialized to None, indicating that the queue is empty.

3. Menu Display: A while True loop displays the menu options: Enqueue, Dequeue, Peek, Display, and Exit.

4. Enqueue (Insertion): A new node is created with the given element. If the queue is empty, both front and rear point to the new node. Otherwise, the new node is linked at the rear, and rear is updated.

.

5. Dequeue (Deletion): If the queue is empty, the program displays "Queue Underflow". Otherwise, the front element is deleted by updating front = front.next. If the queue becomes empty, rear is also set to None.

6. Peek Operation: The program displays the front element using front.data without removing it.

7. Display Operation: A temporary variable temp starts at front and traverses each node using temp = temp.next until it reaches None, printing all elements.

8. Exit Operation: When the user selects option 5, the break statement terminates the loop.

9. Invalid Choice: If the user enters an option other than 1 to 5, the program displays "Invalid choice".

Error Table:

S.No	Error Obtained	Rectification
01	No Errors	----------

.

6b)Aim:To implement circular queue using array

Program:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, data):
        # Check if queue is full
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow. Queue is full.")
            return

        # First element
        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
```

```
        self.rear = (self.rear + 1) % self.size

    self.queue[self.rear] = data
    print(data, "inserted into the circular queue.")

# Dequeue operation
def dequeue(self):
    if self.front == -1:
        print("Queue Underflow. Queue is empty.")
        return

    data = self.queue[self.front]

    # Only one element is present
    if self.front == self.rear:
        self.front = -1
        self.rear = -1
    else:
        self.front = (self.front + 1) % self.size

    print(data, "deleted from the circular queue.")
```

.

```
# Peek operation
def peek(self):
    if self.front == -1:
        print("Queue is empty.")
    else:
        print("Front element is:", self.queue[self.front])

# Display operation
def display(self):
    if self.front == -1:
        print("Queue is empty.")
    else:
        print("The elements of the queue are:")

        i = self.front

        while True:
            print(self.queue[i])

            if i == self.rear:
```

.

```
            break

        i = (i + 1) % self.size

# Count operation
def count(self):
    if self.front == -1:
        print("Number of elements: 0")
    else:
        count = 0
        i = self.front

        while True:
            count += 1

            if i == self.rear:
                break

            i = (i + 1) % self.size

        print("Number of elements:", count)
```

.

```
# Main program
size = int(input("Enter the size of the circular queue: "))

q = CircularQueue(size)

while True:
    print("\n--- CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        q.enqueue(data)
```

.

```
elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()

elif choice == 5:
    q.count()

elif choice == 6:
    print("Program ended.")
    break

else:
    print("Invalid choice.")
```

.

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/W
EEK-6/queueCIrcukar.py
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 23
23 inserted into the circular queue.

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 27
27 inserted into the circular queue.

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 07
7 inserted into the circular queue.

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice:
```

1. Node Class: Creates a node with two parts: data and a pointer to the next node.

2. Constructor (_init_): Initializes the front and rear pointers as None, indicating that the queue is empty.

3. Enqueue Operation: Creates a new node and inserts it at the rear end of the queue. The rear node points back to the front node to maintain the circular structure.

.

4. Dequeue Operation: Deletes the element from the front of the queue. If only one node is present, both front and rear are set to None.

5. Peek Operation: Displays the front element without removing it from the queue.

6. Display Operation: Traverses the linked list from front to rear and displays all elements until it reaches the front node again.

7. Main Program: Displays a menu with Enqueue, Dequeue, Peek, Display, and Exit options. The user selects an operation to perform.

Error Table:

S.No	Errors obtained	Rectification
01	Attribute Error	Maintain proper indentation in Python code
02	Recursion Error	Check the node attributes and function names
03	Type Error	Enter valid integer values for tree operations.

.

WEEK-7

7a) Aim: Python program to implement binary tree using array, linked list with all appix operations (Insertion,deletion,traversal,display,searching).

Program:(Using Array):

```
class BinaryTree:
    def __init__(self):
        self.tree = []

    def insert(self, value):
        self.tree.append(value)
        print("Inserted:", value)

    def delete(self, value):
        if value in self.tree:
            i = self.tree.index(value)
            self.tree[i] = self.tree[-1]
            self.tree.pop()
            print("Deleted:", value)
        else:
```

```
        print("Element not found")

def search(self, value):
    if value in self.tree:
        print("Element found")
    else:
        print("Element not found")

def display(self):
    print("Tree:", self.tree)

def inorder(self, i=0):
    if i < len(self.tree):
        self.inorder(2*i+1)
        print(self.tree[i], end=" ")
        self.inorder(2*i+2)

def preorder(self, i=0):
    if i < len(self.tree):
```

```
        print(self.tree[i], end=" ")
        self.preorder(2*i+1)
        self.preorder(2*i+2)

    def postorder(self, i=0):
        if i < len(self.tree):
            self.postorder(2*i+1)
            self.postorder(2*i+2)
            print(self.tree[i], end=" ")


bt = BinaryTree()

while True:
    print("\n1.Insert 2.Delete 3.Search")
    print("4.Display 5.Inorder 6.Preorder")
    print("7.Postorder 8.Exit")

    ch = int(input("Enter choice: "))
```

.

```
if ch == 1:
    bt.insert(int(input("Enter value: ")))

elif ch == 2:
    bt.delete(int(input("Enter value: ")))

elif ch == 3:
    bt.search(int(input("Enter value: ")))

elif ch == 4:
    bt.display()

elif ch == 5:
    bt.inorder()
    print()

elif ch == 6:
    bt.preorder()
```

```
        print()

    elif ch == 7:
        bt.postorder()
        print()

    elif ch == 8:
        break

    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/WEEK-6/queueCIrcukar.py
PS C:\Users\Sakate\OneDrive\Desktop\DSA> C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/WEEK-6/WEEK-7/binary.py
1.Insert  2.Delete  3.Search
4.Display  5.Inorder  6.Preorder
7.Postorder  8.Exit
Enter choice: 1
Enter value: 23
Inserted: 23

1.Insert  2.Delete  3.Search
4.Display  5.Inorder  6.Preorder
7.Postorder  8.Exit
Enter choice: 1
Enter value: 27
Inserted: 27

1.Insert  2.Delete  3.Search
4.Display  5.Inorder  6.Preorder
7.Postorder  8.Exit
Enter choice: 1
Enter value: 7
Inserted: 7

1.Insert  2.Delete  3.Search
4.Display  5.Inorder  6.Preorder
7.Postorder  8.Exit
Enter choice: 4
Tree: [23, 27, 7]

1.Insert  2.Delete  3.Search
4.Display  5.Inorder  6.Preorder
7.Postorder  8.Exit
Enter choice:
```

Explanation:

1. Array creation: tree = [None] * 100 creates an array with 100 positions to store the tree elements. The variable n keeps track of the number of elements.

2. Insertion: The insert(x) function inserts the given element at the next available array position and increases n by one.

3. Deletion: The delete(x) function searches for the given element, replaces it with the last element in the array and removes the last element.
4. Searching: The search(x) function checks whether the given element exists in the array. If found, it displays the index; otherwise, it displays "Element not found".

.

5. Inorder traversal: The inorder(i) function visits the left subtree, root and right subtree in that order.

6. Preorder traversal: The preorder(i) function visits the root, left subtree and right subtree in that order.

7. Postorder traversal: The postorder(i) function visits the left subtree, right subtree and root in that order.

8. Display: The display() function prints all stored elements along with their array indices.

9. Menu-driven operations: The while True loop displays a menu and allows the user to select an operation. The program continues until the user selects Exit.

Array index formulas For a

node at index i:

- Left child: 2*i + 1
- Right child: 2*i + 2
- Parent: (i-1)//2 Error Table:

S.No	Errors Obtained	Rectification
01	IndentationError: expected an indented block	Correct the indentation of statements inside functions and loops.
02	IndexError: list index out of range	Check array boundaries before accessing an index.

.

7b)Program (using linked list):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    # Insertion
    def insert(self, data):
        new = Node(data)

        if self.root is None:
            self.root = new
            return
```

.

```
    q = [self.root]

    while q:
        temp = q.pop(0)

        if temp.left is None:
            temp.left = new
            return
        else:
            q.append(temp.left)

        if temp.right is None:
            temp.right = new
            return
        else:
            q.append(temp.right)

# Searching
def search(self, root, key):
```

.

```
    if root is None:
        return False

    if root.data == key:
        return True

    return (self.search(root.left, key) or
            self.search(root.right, key))

# Deletion
def delete(self, key):
    if self.root is None:
        print("Tree is empty")
        return

    q = [self.root]
    target = None
    last = None
    parent = None
```

.

```
    while q:
        temp = q.pop(0)

        if temp.data == key and target is None:
            target = temp

        if temp.left:
            parent = temp
            q.append(temp.left)

        if temp.right:
            parent = temp
            q.append(temp.right)

        last = temp

    if target is None:
        print("Element not found")
        return
```

.

```
    if target == last:
        if target == self.root:
            self.root = None
        elif parent.left == last:
            parent.left = None
        else:
            parent.right = None
    else:
        target.data = last.data

        if parent.left == last:
            parent.left = None
        else:
            parent.right = None

    print("Deleted:", key)

# Inorder
def inorder(self, root):
    if root:
```

.

```
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

# Preorder
def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder
def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

# Display
def display(self):
```

```
    .
        if self.root is None:
            print("Tree is empty")
            return

        q = [self.root]

        while q:
            temp = q.pop(0)
            print(temp.data, end=" ")

            if temp.left:
                q.append(temp.left)

            if temp.right:
                q.append(temp.right)


bt = BinaryTree()

while True:
```

.

```
print("\n1.Insert")
print("2.Delete")
print("3.Search")
print("4.Inorder")
print("5.Preorder")
print("6.Postorder")
print("7.Display")
print("8.Exit")

ch = int(input("Enter choice: "))

if ch == 1:
    bt.insert(int(input("Enter value: ")))

elif ch == 2:
    bt.delete(int(input("Enter value: ")))

elif ch == 3:
    key = int(input("Enter value: "))
    if bt.search(bt.root, key):
```

.

```
        print("Element found")
    else:
        print("Element not found")

elif ch == 4:
    bt.inorder(bt.root)
    print()

elif ch == 5:
    bt.preorder(bt.root)
    print()

elif ch == 6:
    bt.postorder(bt.root)
    print()

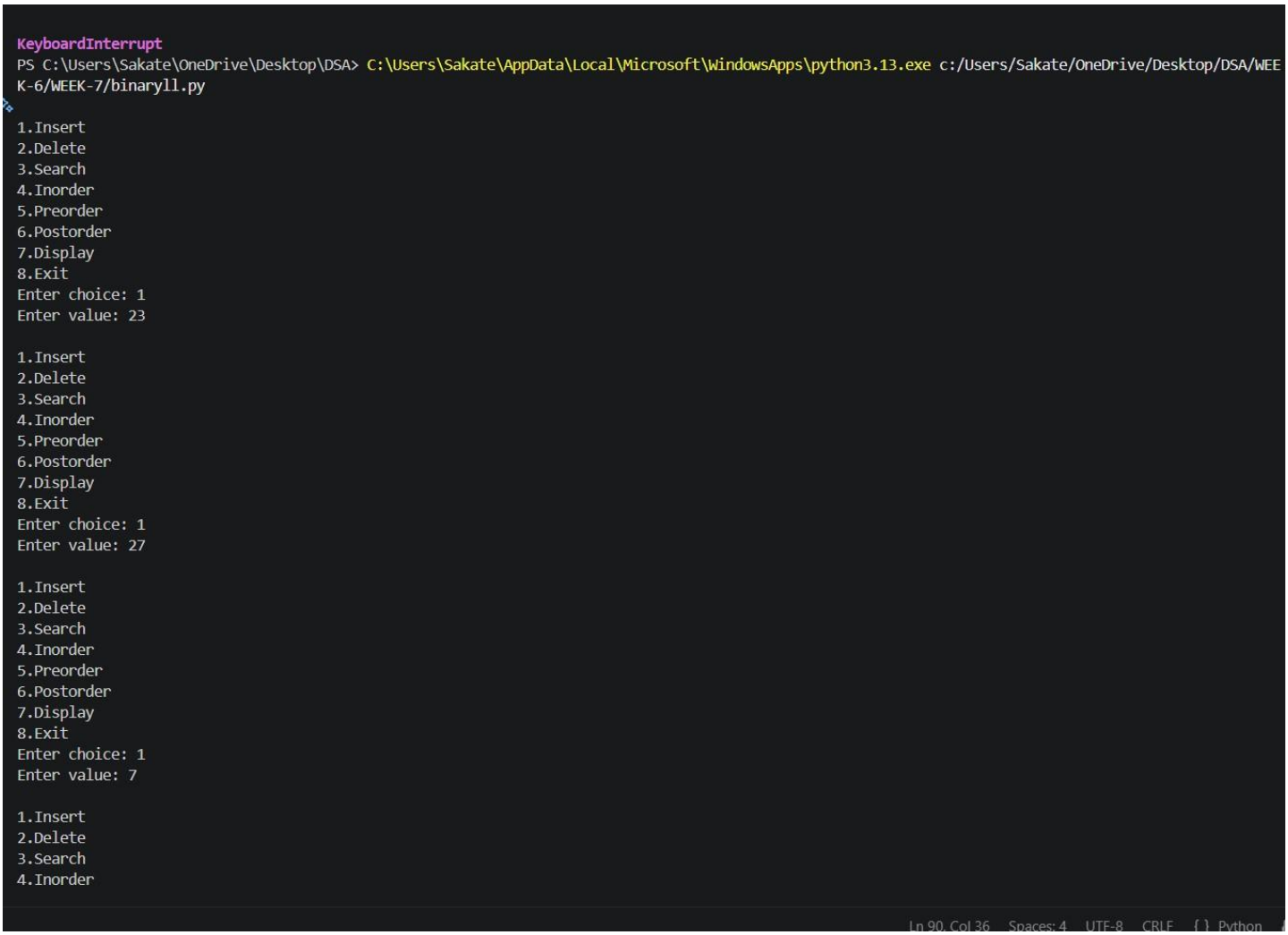
elif ch == 7:
    bt.display()
    print()
```

.

```
elif ch == 8:
    break

else:
    print("Invalid choice")
```

Output:



Explanation:

1. Node creation: The Node class creates a node containing data, left and right. Initially, both child references are None.

2. Root initialization: The variable root stores the address (reference) of the root node. Initially, the tree is empty.

3. Insertion: The insert(x) function creates a new node. If the tree is empty, the new node becomes the root. Otherwise, it uses a queue to find the first available position and inserts the node there.

4. Searching: The search(x) function visits nodes level by level using a queue. It compares each node's data with the required value and displays whether the element is found.

5. Deletion: The delete(x) function finds the node to delete and the deepest, rightmost node. It copies the deepest node's data into the target node and removes the deepest node.

6. Inorder traversal: The inorder(node) function visits the left subtree, root and right subtree.

7. Preorder traversal: The preorder(node) function visits the root, left subtree and right subtree.
8. Postorder traversal: The postorder(node) function visits the left subtree, right subtree and root.

9. Display: The display() function uses a queue to print all nodes level by level.

10. Menu-driven operations: The while True loop allows the user to select insertion, deletion, searching, traversal, display or exit.

- Inorder: 2 1 3

- Preorder: 1 2 3
- Postorder: 2 3 1
- Level-order display: 1 2 3 Error Table:

S.No	Errors Obtained	Rectification
01	NameError: name 'root' is not defined	Initialize root = None before using it.
02	ValueError: invalid literal for int()	Enter a valid integer when prompted for input.

.

WEEK-8

8a)AIM: Write a python program to implement Binary Search tree with its operation.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None



class BST:
    def __init__(self):
        self.root = None

    # Insertion
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
```

```
        root.left = self.insert(root.left, data)
    else:
        root.right = self.insert(root.right, data)

    return root

# Searching
def search(self, root, key):
    if root is None or root.data == key:
        return root

    if key < root.data:
        return self.search(root.left, key)

    return self.search(root.right, key)

# Find minimum value
def minimum(self, root):
    while root.left:
        root = root.left
    return root
```

.

```
# Deletion
def delete(self, root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = self.delete(root.left, key)

    elif key > root.data:
        root.right = self.delete(root.right, key)

    else:
        if root.left is None:
            return root.right

        elif root.right is None:
            return root.left

        temp = self.minimum(root.right)
        root.data = temp.data
        root.right = self.delete(root.right, temp.data)
```

```
    return root

# Inorder
def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

# Preorder
def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder
def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")
```

```
    .

    # Display
    def display(self, root):
        if root:
            print(root.data, end=" ")
            self.display(root.left)
            self.display(root.right)


bst = BST()

while True:
    print("\n1.Insert 2.Delete 3.Search")
    print("4.Inorder 5.Preorder 6.Postorder")
    print("7.Display 8.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        bst.root = bst.insert(bst.root, x)
```

.

```
elif ch == 2:
    x = int(input("Enter value: "))
    bst.root = bst.delete(bst.root, x)

elif ch == 3:
    x = int(input("Enter value: "))
    if bst.search(bst.root, x):
        print("Element found")
    else:
        print("Element not found")

elif ch == 4:
    bst.inorder(bst.root)
    print()

elif ch == 5:
    bst.preorder(bst.root)
    print()

elif ch == 6:
    bst.postorder(bst.root)
    print()
```

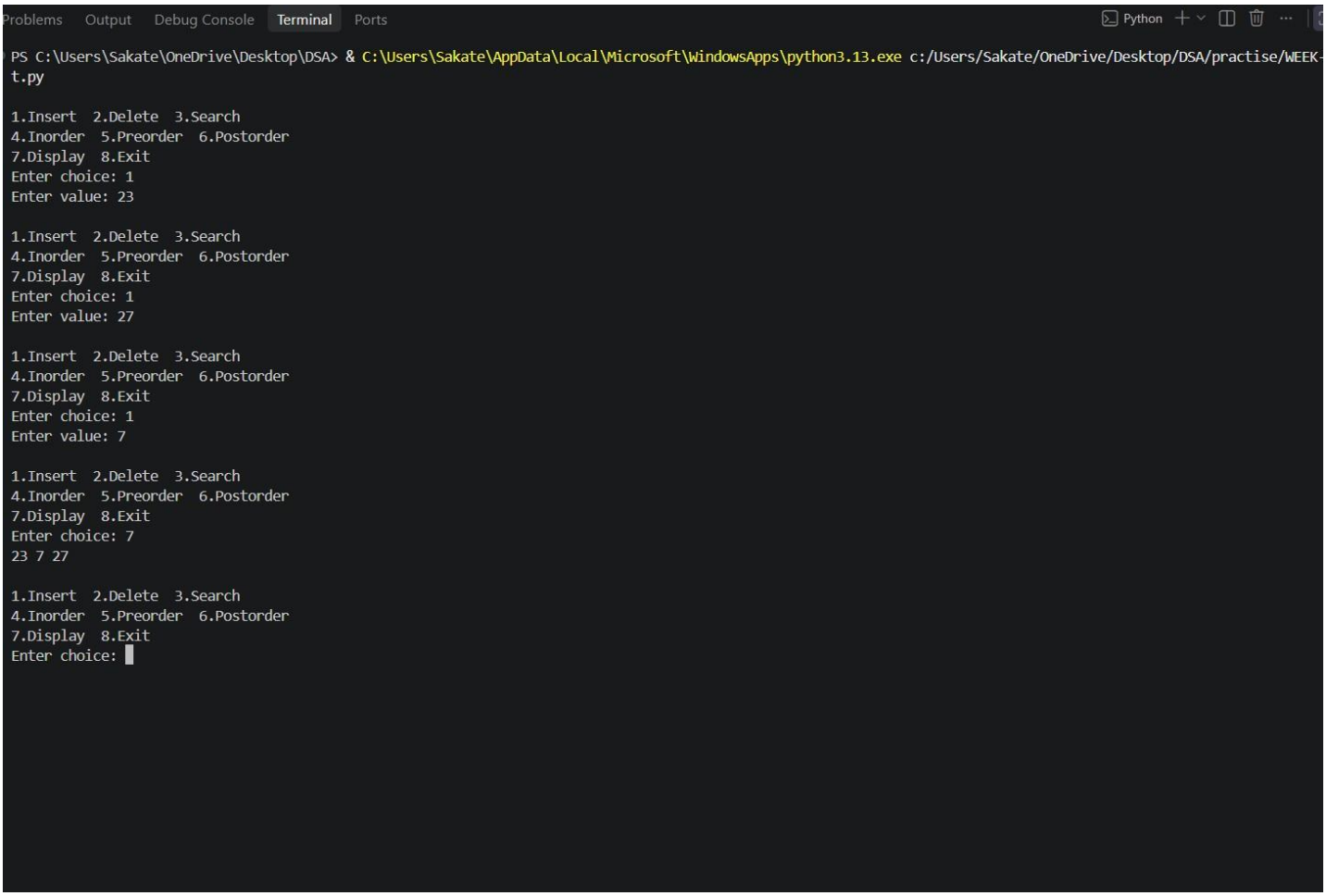
.

```
elif ch == 7:
    bst.display(bst.root)
    print()

elif ch == 8:
    break

else:
    print("Invalid choice")
```

Output:



Explanation:

1. Node Creation: The Node class is used to create a new node containing data, left pointer and right pointer.

2. BST Class: The BST class contains all the operations performed on the Binary Search Tree.

3. Insertion: The insert() function inserts a new node into the tree. If the value is smaller than the root, it is inserted on the left side; otherwise, it is inserted on the right side.

.

4. Searching: The search() function searches for a given element. It compares the key with the root and moves either left or right until the element is found.
5. Deletion: The delete() function removes a node from the tree. It handles three cases: a node with no children, one child, or two children.

6. Find Minimum: The find_min() function finds the smallest value in the right subtree. It is used during deletion when a node has two children.

7. Inorder Traversal: The inorder() function visits the left subtree, root node and right subtree. It displays the elements in ascending order.

8. Preorder Traversal: The preorder() function visits the root node, left subtree and right subtree.

9. Postorder Traversal: The postorder() function visits the left subtree, right subtree and root node.

10. Menu-Driven Operations: A while loop displays the menu and allows the user to select insertion, searching, deletion, traversal or exit.

11. Exit: When the user selects option 7, the break statement terminates the

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Recursion Depth Error (Recursion Error):** Deep or skewed trees exceed Python's default recursion limit (usually 1000 calls) during recursive traversals (inorder, preorder, postorder).	Rewrite traversal functions iteratively using an explicit stack, or increase the recursion limit using sys.setrecursionlimit().

.

WEEK – 9

Aim:To implement AVL tree and perform the basic operations

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None



def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
```

```
    .
    elif data > root.data:
        root.right = insert(root.right, data)

    return root


def search(root, key):
     if root is None:
        return False

    if root.data == key:
        return True
    elif key < root.data:
        return search(root.left, key)
    else:
        return search(root.right, key)


def inorder(root):
```

```
    .
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)



def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)



def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
```

```
    .
        print(root.data, end=" ")



def find_min(root):
    if root is None:
        return None

    while root.left is not None:
        root = root.left

    return root.data




def find_max(root):
    if root is None:
        return None

    while root.right is not None:
```

```
    .
        root = root.right

    return root.data



def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)

    elif key > root.data:
        root.right = delete(root.right, key)

    else:

        if root.left is None and root.right is None:
```

```
        return None

    # Case 2: Only right child
    elif root.left is None:
        return root.right

    # Case 2: Only left child
    elif root.right is None:
        return root.left

    # Case 3: Two children
    temp = root.right

    while temp.left is not None:
        temp = temp.left

    root.data = temp.data
    root.right = delete(root.right, temp.data)
```

```
    .
    return root



# 9. Count total nodes
def count_nodes(root):
    if root is None:
        return 0

    return 1 + count_nodes(root.left) + count_nodes(root.right)



# 10. Height of BST
def height(root):
    if root is None:
        return 0

    return 1 + max(height(root.left), height(root.right))
```

.

```
# 11. Menu-driven program
root = None


while True:
    print("\n\n--- BST OPERATIONS ---")
    print("1. Insert")
    print("2. Search")
    print("3. Inorder")
    print("4. Preorder")
    print("5. Postorder")
    print("6. Delete")
    print("7. Minimum")
    print("8. Maximum")
    print("9. Count Nodes")
    print("10. Height")
    print("11. Exit")

    choice = int(input("Enter your choice: "))
```

```
.
if choice == 1:
    data = int(input("Enter value: "))
    root = insert(root, data)

elif choice == 2:
    key = int(input("Enter value to search: "))
    if search(root, key):
        print("Element found")
    else:
        print("Element not found")

elif choice == 3:
    inorder(root)
    print()

elif choice == 4:
    preorder(root)
    print()
```

```
.
elif choice == 5:
    postorder(root)
    print()

elif choice == 6:
    key = int(input("Enter value to delete: "))
    if search(root, key):
        root = delete(root, key)
        print("Element deleted")
    else:
        print("Element not found")

elif choice == 7:
    print("Minimum:", find_min(root))

elif choice == 8:
    print("Maximum:", find_max(root))

elif choice == 9:
```

```
.
        print("Total nodes:", count_nodes(root))

    elif choice == 10:
        print("Height:", height(root))

    elif choice == 11:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/On
t.py

--- BST OPERATIONS ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Delete
7. Minimum
8. Maximum
9. Count Nodes
10. Height
11. Exit
Enter your choice: 1
Enter value: 23


--- BST OPERATIONS ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Delete
7. Minimum
8. Maximum
9. Count Nodes
10. Height
11. Exit
Enter your choice: 1
Enter value: 27


--- BST OPERATIONS ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
```

Ln 197, Col 32 Spaces: 4 UTF

Explanation:

1. Node Creation: A Node class is created to store the key value, left child, right child and height of each node.

2. AVL Tree Class: The AVLTree class contains all the functions required to perform AVL tree operations.

.

3. Height Calculation: The height() function calculates the height of a node. If the node is empty, it returns 0.
4. Balance Factor: The balance() function calculates the difference between the heights of the left and right subtrees. The balance factor must be -1, 0 or +1.

5. Right Rotation: The rightRotate() function performs right rotation to balance the tree when the left subtree becomes heavy.

6. Left Rotation: The leftRotate() function performs left rotation to balance the tree when the right subtree becomes heavy.

7. Insertion: The insert() function adds a new element following Binary Search Tree rules and checks the balance factor to perform the required rotations.

8. Minimum Value Node: The minValueNode() function finds the smallest value in the right subtree. It is mainly used during deletion.

9. Deletion: The delete() function removes the specified element and updates the height and balance factor of the remaining nodes.

10. Searching: The search() function searches for an element by comparing it with the root node and moving towards the left or right subtree.

11. Inorder Traversal: The inorder() function visits the left subtree, root node and right subtree. It displays elements in sorted order.

12. Preorder Traversal: The preorder() function visits the root node, left subtree and right subtree.

13. Postorder Traversal: The postorder() function visits the left subtree, right subtree and root node.

.

14. Menu-Driven Operations: The main program displays a menu to perform insertion, deletion, searching and traversal operations based on the user's choice.

15. Exit: The program terminates when the user selects option 7.

Error Table:

S.No	Errors obtained	Rectification
01	Attribute Error	Maintain proper indentation in Python code
02	Recursion Error	Check the node attributes and function names
03	Type Error	Enter valid integer values for tree operations.

.

WEEK-10

Aim: To implement hashing using separate chaining , linear and quadratic probing

Program:(Using array):

```
size = 7
table = [[] for i in range(size)]

def insert(key):
    index = key % size
    table[index].append(key)

def search(key):
    index = key % size
    if key in table[index]:
        print("Element found")
    else:
        print("Element not found")

def delete(key):
    index = key % size
    if key in table[index]:
```

```
    .
        table[index].remove(key)
        print("Deleted")
    else:
        print("Element not found")

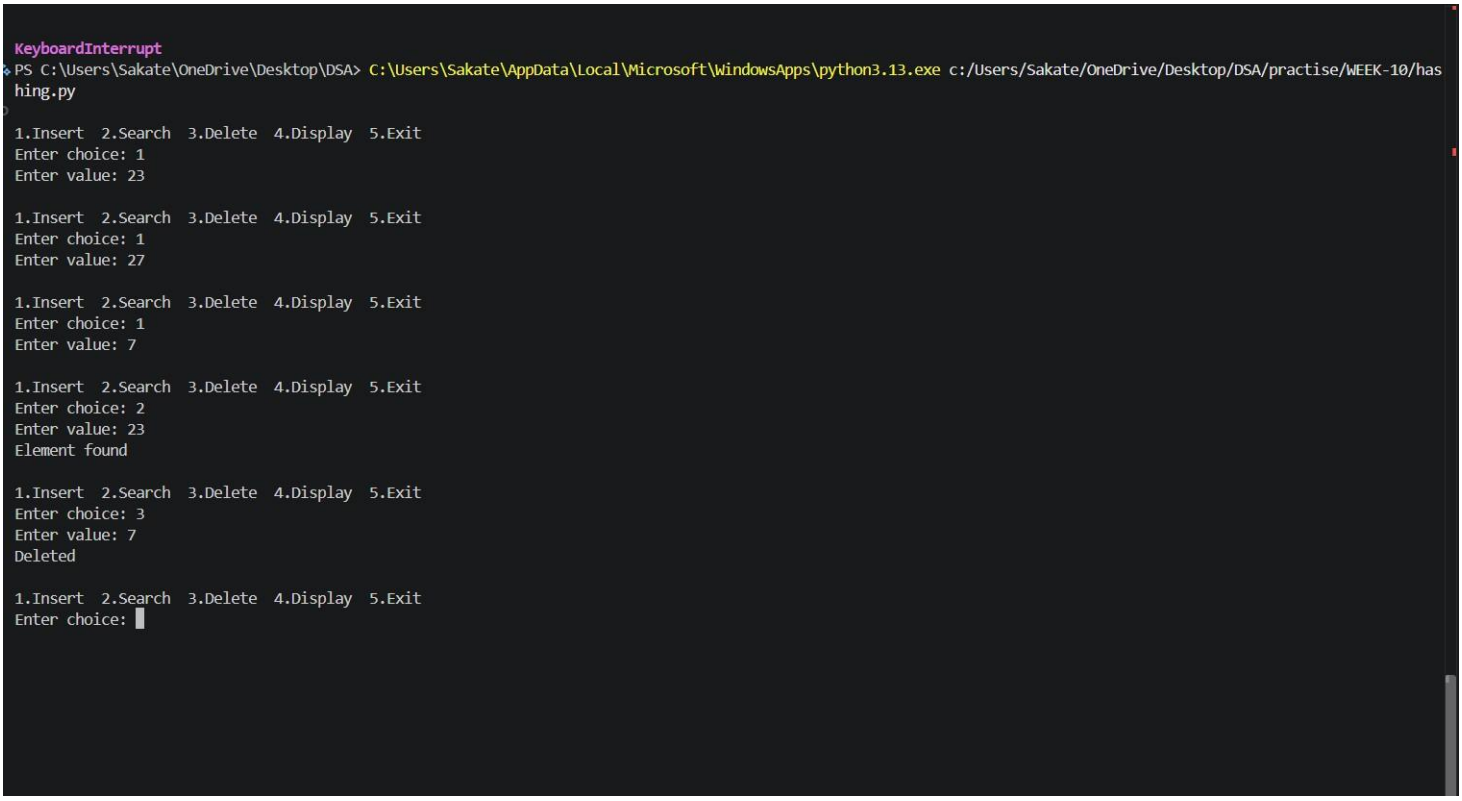
def display():
    for i in range(size):
        print(i, ":", table[i])

while True:
    print("\n1.Insert 2.Search 3.Delete 4.Display 5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        insert(int(input("Enter value: ")))
    elif ch == 2:
        search(int(input("Enter value: ")))
    elif ch == 3:
        delete(int(input("Enter value: ")))
    elif ch == 4:
        display()
    elif ch == 5:
```

```
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Hash Table Creation: A hash table of size 10 is created to store the keys. Separate lists are used for Separate Chaining, Linear Probing and Quadratic Probing.
2. Hash Function: The hash function key % size calculates the index where the key should be stored in the hash table.

3. Separate Chaining: The separate_chaining() function handles collisions by storing multiple keys in a list at the same index.

.

4. Linear Probing: The linear_probing() function checks the next available position whenever a collision occurs. It continues checking until an empty position is found.

5. Quadratic Probing: The quadratic_probing() function handles collisions by checking positions using the square of the probing number (i * i) until an empty position is found.

6. Menu Display: A while loop displays the menu and allows the user to select the required hashing technique.

7. Insertion: The user enters a key, and the selected hashing technique inserts it into the corresponding hash table.

8. Display: The display option prints all three hash tables so that the user can see how the keys are stored.

9. Exit: When the user selects option 5, the break statement terminates the program.

Important Formulae:-

Method	**Formula**
Division	h(x) = x % M
Mid-Square	Square the key and extract middle digits
Digit Folding	Add the separate parts of the key
Multiplication	h(x) = floor(M × fractional(x × C))
Linear Probing	(h(x) + i) % M

.

Quadratic Probing	$(h(x) + i^2)$ % M
Double Hashing	$(h1(x) + i \times h2(x))$ % M
Method	**Formula**
Load Factor	$\alpha = n / M$

Time complexity:-

Operation	**Average Case**	**Worst Case**
Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display	O(n + M)	O(n + M)
Load Factor	O(n + M)	O(n + M)
Rehashing	O(n)	O(n)

SPACE COMPLEXITY: O(n + M)

Error table:

S.No	Errors Obtained	Rectification
1	Index out of range	Used modulo (%) to calculate valid indexes.
2	Collision occurred during	Handled collisions using Separate Chaining.

.

	insertion	
3	Linear probing failed to find an empty position.	Checked the next available index.
4	Quadratic probing collision	Used the quadratic probing formula to find an empty position.

.

10B):Using linked list

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

size = 7
table = [None] * size

def insert(key):
    i = key % size
    new = Node(key)
    new.next = table[i]
    table[i] = new

def   search(key):
    i = key % size
    temp = table[i]
```

```
    .
    while temp:
        if temp.data == key:
            print("Element found")
            return
        temp = temp.next
    print("Element not found")

def delete(key):
    i = key % size
    temp = table[i]
    prev = None

    while temp:
        if temp.data == key:
            if prev:
                prev.next = temp.next
            else:
                table[i] = temp.next
            print("Deleted")
            return
```

```
    .
        prev = temp
        temp = temp.next

    print("Element not found")

def display():
    for i in range(size):
        temp = table[i]
        print(i, end=": ")
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

while True:
    print("\n1.Insert 2.Delete 3.Search 4.Display 5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        insert(int(input("Enter value: ")))
```

.

```
elif ch == 2:
    delete(int(input("Enter value: ")))
elif ch == 3:
    search(int(input("Enter value: ")))
elif ch == 4:
    display()
elif ch == 5:
    break
else:
    print("Invalid choice")
```

Output:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/practise/WEEK-10/
ashingll.py
1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 1
Enter value: 27

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 1
Enter value: 7

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 4
0: 7 -> None
1: None
2: 23 -> None
3: None
4: None
5: None
6: 27 -> None

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 1
Enter value: 48

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 1
Enter value: 40

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 1
Enter value: 2327

1.Insert 2.Delete 3.Search 4.Display 5.Exit
Enter choice: 4
0: 7 -> None
1: None
2: 23 -> None
3: 2327 -> None
4: None
5: 40 -> None
6: 48 -> 27 -> None
```

Explanation:

1. Node Creation: The Node class is used to create a node containing data and a next pointer that stores the address of the next node.
2. Hash Table Creation: The HashTable class creates a hash table of size 10 using an array of linked-list pointers.

3. Initialization: The __init__() function initializes the hash table with all positions set to None.

4. Insertion: The insert() function calculates the index using the hash function key % size and inserts the key into the corresponding linked list.

5. Collision Handling: If two or more keys get the same index, they are stored in the same linked list. This method is called Separate Chaining.

.

6. Searching: The search() function searches for a given key by traversing the linked list at the calculated index.

7. Display: The display() function displays all the hash table indexes and their corresponding linked-list elements.

8. Menu-Driven Operations: A while loop displays the menu and allows the user to select insertion, searching, display or exit.

9. Exit: When the user selects option 4, the break statement terminates the program.

Important Formulae:-

Method	**Formula**
Division	$h(x) = x \% M$
Mid-Square	Square the key and extract middle digits
Digit Folding	Add the separate parts of the key
Multiplication	$h(x) = \text{floor}(M \times \text{fractional}(x \times C))$
Linear Probing	$(h(x) + i) \% M$
Quadratic Probing	$(h(x) + i^2) \% M$
Double Hashing	$(h1(x) + i \times h2(x)) \% M$
Load Factor	$\alpha = n / M$

Time complexity:-

.

Operation	Average Case	Worst Case
Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display	O(n + M)	O(n + M)
Load Factor	O(n + M)	O(n + M)
Rehashing	O(n)	O(n)

SPACE COMPLEXITY: O(n + M)

Error Table:

S.No	Errors Obtained	Rectification
01	Attribute Error	Check the linked list node attributes properly.
02	Index Error	Check the hash table index before accessing it.